COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	10/07/2026	**DATE SUBMITTED:**	10/09/2026
NAME:	Juan Paulo C. Lara Mencidor, Renverlanz D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11.1 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

What procedures did you perform?

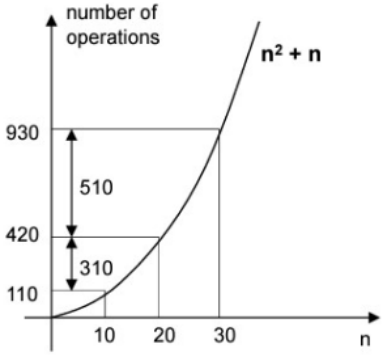
ILO A
Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

Performing worst case analysis:
Let m = the length of array x.
Let n = the length of array y.
The loop in diff repeats n times.
Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: **m x n or mn**

Provide an analysis of the graph for the worse case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Analysis:

(Lara)

This graph represents the growth in the worst-case number of operations by the quadratic function n^2+n. Compared to previous graphs which have a straight line, this graph curves upward so the number of operations grows faster than the given size of arrays. Testing this function using the given n values in the table, starting from 0 to 10 elements adds 110 operations, 10 to 20 adds 420 operations, and 20 to 30 adds 930 operations. This means that the growth is not linear since each increase contains a sum of the squared input value and the base input value. A notable trait in quadratic growth is when the given value is doubled and the resulting value is multiplied by roughly four times from the previous result. This is especially noticeable when comparing the number of operations for n size 10 and 20, which shows 110 and 420. As n increases, the n^2 greatly outnumbers n in the equation. Therefore, this algorithm is unsuitable for larger inputs due to the exponential increase in the number of operations as the number of elements increases.

(Mencidor)

This graph shows how the number of operations grows using the formula n^2 + n. Unlike a straight line graph, this one curves upward, meaning operations increase much faster as the input size grows.
For example, going from 0 to 10 elements adds 110 operations but going from 20 to 30 elements adds 930 operations. This happens because doubling the input size roughly quadruples the total operations. As it gets larger, the n^2 part dominates the equation.

ILO B

Code

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <numeric>

class Student{
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    //constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v){
    }

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator ==(const Student& p) const {
    return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator< (const Student& p) const {
    return this->name < p.name;
    }

    bool operator> (const Student& p) const {
    return this->name > p.name;
    }
};
```

```cpp
auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());

    // the IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}
```

```cpp
bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();
    while (true)
    {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

        // Return true if the Student is found in the sequence and
        // he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false)
        return true;
        else if (mid_element == P && mid_element.is_vaccinated() == true)
        return false;
        else if (mid_element > P)
        std::advance(last, -mid_element_index);
        if (mid_element < P)
        std::advance(first, mid_element_index);

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1)
        return true;
    }
}
```

```cpp
void search_test(int size, Student p) {
    std::vector<Student> people;

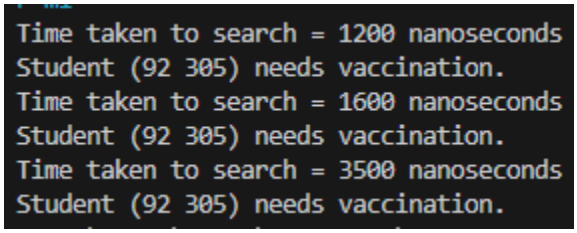
    // Create a list of random people
    for (auto i = 0; i < size; i++)
        people.push_back(generate_random_Student(size));

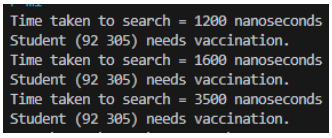
    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin =
    std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();
    std::cout << "Time taken to search = " <<
        // std::chrono::duration_cast<std::chrono::microseconds>
        std::chrono::duration_cast<std::chrono::nanoseconds>
        (end - begin).count() << " nanoseconds" << std::endl;
    if (search_result)
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
        << ") " << "needs vaccination." << std::endl;
    else
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
        << ") " << "does not need vaccination." << std::endl;
}
```

```cpp
int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

Output	

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1,000	1.2 microseconds		This is the baseline time of execution which includes the time taken by the algorithm and the input size.
10,000	1.6 microseconds		The Execution Time has increased by 0.4.
100,000	3.5 microseconds		The Execution Time has increased by roughly 2x from the execution of N size 10,000..

Analysis:
(Mencidor)
The “needs vaccination” function implements a Binary Search algorithm over a sorted vector of *Student* objects. By dividing the search interval in half during each iteration, binary search exhibits a logarithmic theoretical time complexity of “O(log2 N)”. Experimentally, increasing the input size (from 1,000 to 100,000) only increased the execution time to from 1.2 to 3.5 microseconds. This confirms the logarithmic growth behavior. The increased minimal time shows that the binary search scales efficiently for massive datasets compared to linear search methods.

(Lara)
The provided code contains the vaccination function, which uses a binary search algorithm to search for the given list of students that may need vaccination. Since a binary search divides the list in half for each iteration, the theoretical time complexity is $O(\log_2 N)$, which means that increasing the input size by a large magnitude only results in an additional growth in time instead of a proportional growth. The experimental data proves the time complexity, notable with the increase of 1000, 10000, and 100000 only increasing by roughly 0.3 and 0.8 microseconds for each increase.

Table 11-1. Results of Runtime Performance by Comparing Experimental and Theoretical Analysis.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1

Theoretical Analysis

```
Algorithm Unique(A)
  for i = 0 to n-1 do
    for j = i+1 to n-1 do
      if A[i] equals to A[j] then
        return false
  return true
```

This algorithm performs comparisons on every pair of elements to determine if they are unique or a duplicate. It continues comparing each pair until it reaches a duplicate.

Operation Analysis	
Initialization of i	1
Outer loop	n
Inner comparisons	n(n-1)/2
Equal comparison	n(n-1)/2
Return true	1
Return false	1 if the duplicate is found

The equation of this algorithm used to determine the number of pairs is: $T(n) = \frac{n(n-1)}{2}$

Distributing the equation, the result is: $T(n) = \frac{(n^2-n)}{2}$

Taking the dominant term, the resulting time complexity equation is: $T(n) = O(n^2)$

Case Analysis		
Best	When a duplicate is found immediately	O(1)
Worst	When all elements are unique	$O(n^2)$
Average	When it takes many comparisons to find the duplicate	$O(n^2)$

Experimental Analysis
Code Implementation

Code

```cpp
#include <iostream>
#include <vector>
#include <chrono>

bool Unique(std::vector<int>& A) {
    int n = A.size();

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (A[i] == A[j]) {
                return false;
            }
        }
    }

    return true;
}
```

```cpp
void test_unique(int size) {
    std::vector<int> A;

    // Create an array containing unique elements
    for (int i = 0; i < size; i++) {
        A.push_back(i);
    }

    // Start the clock
    std::chrono::steady_clock::time_point begin =
        std::chrono::steady_clock::now();

    bool result = Unique(A);

    // Stop the clock
    std::chrono::steady_clock::time_point end =
        std::chrono::steady_clock::now();

    std::cout << "Array size = " << size << std::endl;
    std::cout << "Time taken = "
        << std::chrono::duration_cast<std::chrono::microseconds>
        (end - begin).count() << " microseconds" << std::endl;

    if (result) {
        std::cout << "All elements are unique." << std::endl;
    } else {
        std::cout << "Duplicate elements found." << std::endl;
    }

    std::cout << std::endl;
}
```

```cpp
int main() {
    test_unique(1000);
    test_unique(5000);
    test_unique(10000);
    test_unique(20000);

    return 0;
}
```

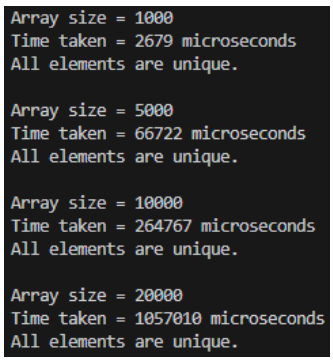
Output

```
Array size = 1000
Time taken = 2679 microseconds
All elements are unique.

Array size = 5000
Time taken = 66722 microseconds
All elements are unique.

Array size = 10000
Time taken = 264767 microseconds
All elements are unique.

Array size = 20000
Time taken = 1057010 microseconds
All elements are unique.
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
1000	2679 microseconds		The algorithm completes almost instantly because the number of comparisons is relatively small.
5000	66722 microseconds		Increasing the input size by 5 times causes execution time to jump roughly 25 times. The algorithm forces a full scan since all elements generated are unique.
10000	264767 microseconds		Doubling the input size from 5,000 to 10,000 increases execution time by roughly 4 times, matching the expected quadratic behavior($2^2 = 4$).
20000	1057010 microseconds		Doubling the input size again to 20,000 causes the execution time to quadruple to over 1 second, demonstrating steep growth.

Analysis and Comparison
(Lara)
The provided algorithm checks if all elements in an array are unique. It contains two nested for loops, which contain an outer loop for selecting an element at index i, and the inner loop checks every next element at

index j. It also contains an equality check for the current index of i and j to determine a duplicate. If no duplicates are found, it returns false. Theoretically, this means that it has a worst-case time complexity of $O(n^2)$, which is a quadratic form, meaning that the resulting time taken by the algorithm increases by approximately four times. With the formulated time complexity equation, the test code proves this with the resulting time of N size 5000, 10000, and 20000 showing a consistent increase of 4 times from the previous input values.

(Mencidor)
The algorithm uses 2 nested loops, which theoretically gives it a worst case time complexity of $O(n^2)$. In the test code, the input array contains only unique numbers, forcing the program to check every possible pair before returning "true". This puts the code directly into its worst case scenario.

Quadratic growth(n^2) means that doubling the input size roughly quadruples the run time which is $2^2 = 4$:
- Doubling from 5,000 to 10,000 increased time by ~3.97 times.
- Doubling from 10,000 to 20,000 increased time by ~3.99 times.

Problem 2

Theoretical Analysis

```
Algorithm rpower(int x, int n):
1 if n == 0 return 1
2 else return x*rpower(x, n-1)

Algorithm brpower(int x, int n):
1 if n == 0 return 1
2 if n is odd then
3   y = brpower(x, (n-1)/2)
4   return x*y*y
5 if n is even then
6   y = brpower(x, n/2)
7   return y*y
```

Operation Analysis for rpower	
Check n == 0	**n + 1**
Recursive call	**n**
Multiplication	**n**
Best case return	**1**

The equation of this function used to determine the number of pairs is: $T(n) = n + 1$
Taking the dominant term, the resulting time complexity equation is: $T(n) = O(n)$

Operation Analysis for brpower	
If n == 0	**1**
If n is odd	**1**
Division	**1**
Multiplication	**1 (if n is even) or 2 (if n is odd)**
Recursive call	**1**

The equation of the recursive call in this function is: $T(n) = O(n/2) + 1$ if n is even.
Dividing this equation by n times to obtain 1 results in: $\log_2 n$
Determining the time complexity, the dominant terms of O(n) and $\log_2 n$ are combined into: O(log n)

Experimental Analysis

Code

```cpp
#include <iostream>
#include <chrono>

long double rpower(long double x, int n) {
    if (n == 0) {
        return 1;
    }

    return x * rpower(x, n - 1);
}

long double brpower(long double x, int n) {
    if (n == 0) {
        return 1;
    }

    if (n % 2 == 1) {
        long double y = brpower(x, (n - 1) / 2);
        return x * y * y;
    }

    long double y = brpower(x, n / 2);
    return y * y;
}
```

```
void test_power(int n) {
    long double x = 1;

    // Start rpower timer
    std::chrono::steady_clock::time_point begin1 =
        std::chrono::steady_clock::now();

    long double result1 = rpower(x, n);

    // Stop rpower timer
    std::chrono::steady_clock::time_point end1 =
        std::chrono::steady_clock::now();

    // Start brpower timer
    std::chrono::steady_clock::time_point begin2 =
        std::chrono::steady_clock::now();

    long double result2 = brpower(x, n);

    // Stop brpower timer
    std::chrono::steady_clock::time_point end2 =
        std::chrono::steady_clock::now();

    std::cout << "n = " << n << std::endl;
    std::cout << "n = " << n << std::endl;

    std::cout << "rpower time = " << std::chrono::duration_cast<std::chrono::nanoseconds>
        (end1 - begin1).count() << " nanoseconds" << std::endl;

    std::cout << "brpower time = " << std::chrono::duration_cast<std::chrono::nanoseconds>
        (end2 - begin2).count() << " nanoseconds" << std::endl;

    std::cout << std::endl;
}
```

```
int main() {
    test_power(10);
    test_power(100);
    test_power(1000);
    test_power(5000);
    test_power(10000);

    return 0;
}
```

Output

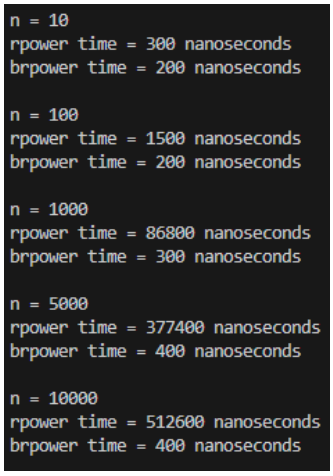
```
n = 10
rpower time = 300 nanoseconds
brpower time = 200 nanoseconds

n = 100
rpower time = 1500 nanoseconds
brpower time = 200 nanoseconds

n = 1000
rpower time = 86800 nanoseconds
brpower time = 300 nanoseconds

n = 5000
rpower time = 377400 nanoseconds
brpower time = 400 nanoseconds

n = 10000
rpower time = 512600 nanoseconds
brpower time = 400 nanoseconds
```

Input Size (N)	Execution Speed	Screenshot	Observation(s)
10	0.3 microseconds (rpower) 0.2 microseconds (brpower)		Both functions run almost instantaneously at a tiny input size.
100	1.5 microseconds (rpower) 0.2 microseconds (brpower)		*rpower* takes 5 times longer as N increases 10 fold, while *brpower* stays constant at 200 ns.
1000	86.8 microseconds (rpower) 0.3 microseconds (brpower)		The execution time of *rpower* grows significantly, whereas *brpower* only increases by 100 ns.
5000	377.4 microseconds (rpower) 0.4 microseconds (brpower)		*rpower* scales proportionally with N, while *brpower* remains practically flat.

10000	512.6 microseconds (rpower) 0.4 microseconds (brpower)		At 10,000, *brpower* is over 1,280 times faster than *rpower.*

Analysis and Comparison
(Lara)
The data from the theoretical and experimental analysis compares the linear and logarithmic scaling of the two algorithms processing the input values. The rpower function takes O(n) operations, which means that the execution time increases proportionally with N. While brpower takes O(n log n) operations, which means that increasing the value of N adds a slow growth in time compared to the rpower function.

(Mencidor)
rpower subtracts 1 from N on every call (n-1). Execution time grows steadily and directly alongside the input size. While, ***brpower*** divides N in half on every call. Execution time grows extremely slowly because the problem size shrinks rapidly.

Conclusion (Lara)
This hands-on activity has introduced the concepts of basic algorithmic analysis by providing the necessary data tables for the expected time complexity equations, common functions used in analysis, and growth rates that show the scale of the growth for each time complexity formulated from the given algorithms, which were shown later in the procedures and supplementary activities. The procedures demonstrated the steps in analyzing algorithms in terms of their number of comparisons, and providing an analysis with the given graph of a quadratic equation along with a table providing the resulting values of the theoretical analysis. The supplementary activities extend this concept to a practical level with working code implementations tested on systems, which add variation in results due to the differences in hardware, operating system, and IDE compiler configurations. Similarly to the previous procedures, it also demonstrated formulations of time complexity equations with the provided codes and implementations to create concrete results and comparisons for theoretical and experimental analysis.

Conclusion (Mencidor)
This activity covered the basics of algorithm analysis by comparing theoretical predictions with real world test results. Through the main procedures, we learned how to count comparisons, build time complexity equations, and plot growth rates using data tables and graphs. The *Supplementary Activities* then applied these concepts to actual code. While real world test times varied depending on the computer's hardware, operating system, and compiler, comparing them to our mathematical formulas clearly showed how theoretical analysis matches up with practical execution.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.